🐍 Before the Course

MARAMA Python Tutorial Series, 2026

> **🎯 The goal**
>
> This guide helps you set up a working Python environment with everything needed for the course. You have two ways to do this: **Google Colab** (recommended, because it runs in your browser) or **your own computer**.

The Course will cover:

Session	Topic
1	Overview of Python and Simple Statistics in Python
2	Data Analysis Techniques and Simple Plotting
3	Python Tools for Atmospheric Science
4	Analysis Workflows and Pipelines

During the course you will need:

- **A laptop and charger.** Windows, Mac or Linux all work.
- **An internet connection.**
- If you are using Google Colab, you also need **a Google account** (a personal one is fine) **and a current web browser** (Chrome, Edge, Firefox or Safari).

Setting up Google Colab

Colab is a free Google service that runs Python in your browser. Nothing to download to disk. It works the same on Windows, Mac and Linux.

1. Go to colab.research.google.com and sign in.

2. Choose **File → New notebook**.
3. **Choose the Python version.** Colab now defaults to Python 3.13, but this course is designed for Python 3.12. Choose **Runtime → Change runtime type**, open the **Runtime Version** dropdown, select **2026.07** (it ships Python 3.12), and click **Save**. Connecting can take 2 to 3 minutes. That is normal.
4. **Check the version.** Copy this into a cell and press **Shift+Enter** to run it:

```
import sys
print("Python", sys.version.split()[0])
print("✅ Correct version." if sys.version_info[:2] == (3, 12) else "❌ Wrong
version. Repeat step 3 and choose a Runtime Version that ships Python 3.12.")
```

1. **Check the packages.** Copy this into a **new cell** and press **Shift+Enter** to run it. It checks whether the Python packages for the course are installed:

```
import importlib
names = ["numpy", "pandas", "geopandas", "matplotlib", "scipy",
         "xarray", "netCDF4", "h5py", "cartopy", "plotly"]
missing = []

for name in names:
    try:
        mod = importlib.import_module(name)
        print(f"OK       {name:12s} {getattr(mod, '__version__', '')}")
    except Exception as err:
        missing.append(name)
        print(f"MISSING  {name:12s} {type(err).__name__}")

print()
if not missing:
    print("✅ Ready. You are set for the course.")
else:
    print("❌ Not ready yet. Missing:", ", ".join(missing))
    print()
    print("To fix it: copy the line below into a NEW cell, run it, then run this
check cell again.")
    print()
    print("!pip install " + " ".join(missing))
```

If the last line says ✅ Ready, you are done.

If it says ❌ Not ready yet, it prints a line that starts with `!pip install`. Copy that whole line into a new cell and run it. Then run the check cell again. The last line should now say ✅ Ready.

What does `!pip install` do?

- It downloads and installs packages from the Python Package Index (PyPI).
- The `!` tells Colab to run it as a computer command, not as Python. Do not remove it.

What you should see while it installs. A few minutes of scrolling text (that is normal). The output ends with ***Successfully installed...*** followed by the package names. Red text in the middle is often only a warning. If Colab shows a **Restart session** button, click it, then run the version check and the package check again.

> 💡 **Colab forgets everything between visits**
>
> Each time you open Colab you get a **fresh computer**, so your installs disappear when the session ends. The Python version may also go back to the default. That is expected. To make this quick every time:
>
> - Save your notebook: **File → Save a copy in Drive**. Name it "MARAMA setup".
> - **About 10 minutes before each session starts,** open that notebook and run the version check. If it says ❌, repeat step 3. Then run the package check and wait for ✅ **Ready**.
> - Colab can disconnect when idle. If it does, choose **Runtime → Restart session** and run the cells again.

If something goes wrong in Colab

Problem	What to try
The version check says ❌ **Wrong version**	Repeat step 3: **Runtime → Change runtime type → Runtime Version → 2026.07 → Save**. Wait for it to connect, then run the version check again.
The **Runtime Version** dropdown is missing, or 2026.07 is not listed	Email us a screenshot of the dropdown (address below).
The package check says ❌ **Not ready yet**	Copy the `!pip install` line it printed into a new cell and run it. Then run the check again.
The install prints a lot of red text	Often only warnings. Look at the last lines. If they say `Successfully installed`, you are fine. If they say `ERROR`, run the line once more. If it fails again, email us the output.
Colab shows "Session crashed" or disconnected	**Runtime → Restart session**, then run the version check and the package check again.
A cell says `ModuleNotFoundError`	Run the package check and follow what it prints.
You cannot sign in to Google, or Colab is blocked at work	Use a personal device, or set up your own computer (next section).
Nothing here helps	Email mroots1@umbc.edu with a screenshot. **Please do this before the course,** not during the session.

If using your own computer

Use this if Colab is blocked for you, if you prefer to work offline, or if you want to keep using Python after the course. Once you finish, your setup stays installed, so you do not repeat it for each session.

Pick **one** method. If you are unsure, use **Method 1 (uv)**.

Python version: use 3.12. The course supports **Python 3.11 and 3.12 only**. Python 3.10 and older are deprecated. Do not use 3.13 or newer. `pyrsig`, the package that downloads EPA and NASA data, has only been vetted on **3.12**, so that is the version we recommend and test. Use 3.11 only if you already have it and cannot change it.

Check which Python you have: run `python --version` (or `python3 --version`). If it shows anything other than 3.11 or 3.12, the steps below set up 3.12 inside a separate environment. They do not change your existing Python.

> **Windows tip:** use **PowerShell**. On Mac and Linux, use **Terminal**.

Words used below. A *terminal* is a window where you type commands. An *environment* is a separate folder of packages, so nothing you do here can break other software on your computer.

Method 1 (recommended): uv

`uv` is a newer, very fast tool that installs Python and packages, manages environments and projects, and keeps things fast and clean. It downloads Python 3.12 for you if you do not have it.

1. Install `uv` from the official installation page. Close and reopen your terminal afterward. - **Mac / Linux:** `curl -LsSf https://astral.sh/uv/install.sh | sh` - **Windows (PowerShell):** `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`
2. Create the environment. Find a good place on your computer and do the following:

```
mkdir marama_python_course && cd marama_python_course
uv venv --python 3.12
```

1. Activate it. - **Mac / Linux:** `source .venv/bin/activate` - **Windows (PowerShell):** `.venv\Scripts\Activate.ps1` - If Windows blocks the script, run `Set-ExecutionPolicy -Scope Process RemoteSigned` and try again.
2. Install the course packages:

```
uv pip install numpy pandas geopandas matplotlib scipy xarray netCDF4 h5py cartopy
plotly jupyterlab
```

1. Start the notebook interface by running `jupyter lab`. Your browser opens. Create a new Python notebook and run the check at the end of this section.

Next time: open a terminal, run `cd marama_python_course`, activate the environment (step 3), then run `jupyter lab`.

Method 2: Python's standard tools (`venv` and `pip`)

This uses only what comes with Python. It is the most universal and the most manual.

1. Install **Python 3.12** from python.org/downloads (choose a 3.12.x release, not the newest version on the front page). **On Windows, tick "Add python.exe to PATH" on the first screen of the installer.**
2. Create the environment. On some Mac/Linux systems the command is `python3`.

```
mkdir marama_python_course && cd marama_python_course
python3.12 -m venv marama_python_course-env
```

1. Activate it. - **Mac / Linux:** `source marama_python_course-env/bin/activate` - **Windows (PowerShell):** `marama_python_course-env\Scripts\Activate.ps1` - If Windows blocks the script, run `Set-ExecutionPolicy -Scope Process RemoteSigned` and try again.
2. Install the course packages:

```
python -m pip install --upgrade pip
python -m pip install numpy pandas geopandas matplotlib scipy xarray netCDF4 h5py
cartopy plotly jupyterlab
```

1. Start the notebook interface by running `jupyter lab`. Your browser opens. Create a new Python notebook and run the check below.

Next time: open a terminal, run `cd marama_python_course`, activate the environment (step 3), then run `jupyter lab`.

Official reference: Python Tutorial: Virtual Environments and Packages.

Check that everything works

Paste this into a notebook cell and run it with **Shift+Enter**:

```
import sys, importlib
print("Python", sys.version.split()[0])
names = ["numpy", "pandas", "geopandas", "matplotlib", "scipy",
         "xarray", "netCDF4", "h5py", "cartopy", "plotly"]
missing = []

for name in names:
    try:
        mod = importlib.import_module(name)
        print(f"OK       {name:12s} {getattr(mod, '__version__', '')}")
    except Exception as err:
        missing.append(name)
        print(f"MISSING  {name:12s} {type(err).__name__}")

print()
if not missing:
    print("✅ Ready. You are set for the course.")
else:
    print("❌ Not ready yet. Missing:", ", ".join(missing))
    print("To fix it, run this in your terminal (with the environment activated),
then run this check again:")
    print()
    print("pip install " + " ".join(missing))
```

Local install troubleshooting

Problem	What to try
`uv` is "not found"	Close and reopen the terminal.
`python --version` shows 3.13 or newer, or 3.10 or older	With uv, make the environment with 3.12 explicitly: `uv venv --python 3.12`. With plain `venv`, install Python 3.12 from python.org and use its full name, such as `py -3.12 -m venv marama_python_course-env` on Windows or `python3.12 -m venv marama_python_course-env` on Mac/Linux.
`python` is "not found" (Windows)	Reinstall and tick **Add python.exe to PATH**, or try `py` instead of `python`.
A package fails to install	Read the last lines of the error and email them to us. Colab is a good fallback in the meantime.
`ModuleNotFoundError` for a package you installed	You are probably in a different environment. Activate the course environment, then run `jupyter lab` from the same window.
Packages install but the notebook cannot see them	Start `jupyter lab` from the activated environment, not a different Python.
Your work computer will not let you install software	Use Colab (see **Setting up Google Colab**). It needs only a browser.

Questions? Email mroots1@umbc.edu